

从 riscv-opcodes 到端到端指令支持

YSX auto-td 自动 TableGen 生成技术报告

昇腾 950 CCU 编译器·内部技术资料

2026-06-15 · 分支 ysx-tiny-fv-ccu (基线 f5fd71c)

本报告不含性能测试数据。所有代码/YAML/TableGen 片段均标注其在工程中的文件与行号区间，行号以本分支当前树为准；代码清单为 ASCII 源逐字呈现，中文注解一律置于清单外的正文，凡为版面计裁剪者均标注“节选”。

摘要

我们为昇腾 950 CCU 实现的基础编译器，在 ISA 裁剪后以标量 rv64ima 为基线。为支撑数据加载与 reduce 等向量操作，需要在此基线上补回一小批 RVV 指令。本工作不沿用“为每条指令手写嵌套 TableGen 指令类”的传统路径，而是构建了 **auto-td** 生成系统：每条新指令由一个**结构化 YAML 清单**声明其身份，其**编码 100% 来自 opcode 数据库**（标准指令取自外部权威 riscv-opcodes，自定义指令取自本团队维护的 ysx-opcodes）、**操作数与副作用 100% 来自共享 taxonomy**，由 Python 生成器产出真正的指令 TableGen 记录、审计 manifest，以及（仅对显式标记 builtin.codegen: true 的证明 API）Clang builtin TD、LLVM intrinsic TD 与 CGBuiltin 派发片段。当前切片覆盖 50 条指令（15 tiny-F + 35 tiny-V，其中 49 条来自 riscv-opcodes、1 条 yushuxin.vfexp 来自 ysx-opcodes）。

需要诚实界定端到端的广度：50 条中**只有 8 条**(vadd.vv/vsub.vv/vmul.vv/vredsum.vs/vfredusum.vs/vr带生成的 builtin+intrinsic+CGBuiltin 并具备从 C 到目标码的路径，其余 42 条仅到“指令记录 + MC 编码”层；且这 8 条的 intrinsic→ 机器下降仍是**手写 C++**，用户头文件 ysx_vector.h 也是手写的（生成器不产 .h）。

本报告的核心是一组**新旧对比**：传统上端到端引入一条向量指令需要在多个文件、约六层 TableGen 类/多类之间铺陈编码位、操作数、调度、pseudo、pattern、intrinsic、builtin；而在 auto-td 下，新增一条指令在常见情形下**只需新增一个 YAML 文件**（必要时加一个 taxonomy 类目）。我们同时对一个常被忽略的命题给出**诚实分层判断**：上游 TableGen 的“人为知识”中，**编码/操作数位置/汇编-反汇编层**确实可由 opcode 数据库机械派生，但**语义/ISel/调度/intrinsic 类型/ABI**是真正的人为设计，无法靠解析 riscv-opcodes（或散文式 riscv-isa-manual）立刻得到。auto-td 的创新不是“消灭人为知识”，而是**把机械层与设计层分离**。最后，我们补强了守护，使“新增 tiny-F/V 指令记录必须经 auto-td”这一承诺，在其覆盖的形态内，从约定变为构建/CI 门禁。

1 引言

在 LLVM 后端引入一条新指令的工程成本，主要不在“这条指令本身”，而在它必须被同时表达成多种彼此独立的事实：机器码编码、汇编/反汇编语法、操作数与寄存器类、副作用与调度、选择 DAG 模式、intrinsic 原型、前端 builtin 与头文件。传统 RISC-V 后端用一套深层 TableGen 类/多类库来复用这些表达，但代价是：**加一条指令前，先要读懂并挑对这套类层级**；遇到库里没有的新形态，还要先扩库。

本工作提出的问题是：这些表达里，有多少其实是可由权威数据源机械派生的，有多少是真正的人为设计？贡献为：(1) 一个把两者显式分离的 auto-td 生成系统；(2) 一条自定义指令 yushuxin.vfexp 的端到端存在性证明；(3) 一道把“无手写指令 TD”承诺机械化的源端守护；(4) 对上述命题的诚实分层判断。全文以“旧路 vs 新路”为主线（第 3-6 节）。图 1、2、3 分别给出数据流、新旧对比与守护关系。

2 背景：上游 RVV 指令的完整 bring-up 路径

以一条最普通的整数向量加法 `vadd.vv` 为例，上游 LLVM/Clang 把它表达成以下几类事实（以下 TableGen 片段均为节选）。

(a) **编码与汇编（格式类，手工誊写编码位）**。编码位被手写进格式类的 `let Inst{}`。注意末行 `RVVConstraint` 是设计层信息、非编码，opcode DB 里没有：

```
class RVInstVV<bits<6> funct6, RISCVVFormat opv, dag outs, dag ins,
            string opcodestr, string argstr>
  : RVInst<outs, ins, opcodestr, argstr, [], InstFormatR> {
  bits<5> vs2; bits<5> vs1; bits<5> vd; bit vm;
  let Inst{31-26} = funct6;      let Inst{25}      = vm;
  let Inst{24-20} = vs2;         let Inst{19-15} = vs1;
  let Inst{14-12} = opv.Value;   let Inst{11-7}  = vd;
  let Inst{6-0}   = OPC_OP_V.Value;
  let Uses = [VL, VTYPE];
  let RVVConstraint = VMConstraint;
}
```

llvm/lib/Target/RISCV/RISCVInstrFormatsV.td:107-125 (class RVInstVV, 节选)

(b) **指令记录（嵌套多类）**。实际指令经多类库实例化，多类之间层层 mix-in：

```
multiclass VALU_IV_V_X_I<string opcodestr, bits<6> funct6>
  : VALU_IV_V<opcodestr, funct6>, VALU_IV_X<opcodestr, funct6>,
    VALU_IV_I<opcodestr, funct6>;
...
defm VADD_V : VALU_IV_V_X_I<"vadd", 0b0000000>;
```

llvm/lib/Target/RISCV/RISCVInstrInfoV.td:612 与:1137

(c) **Pseudo 与选择模式**。另有一套 `VPpseudo/VPat` 多类，规模庞大（`RISCVInstrInfoVPseudos.td` 共 7332 行）；一条 `defm` 会展开出完整的 **LMUL/掩码/policy 矩阵**：

```
multiclass VPseudoBinaryV_VV<LMULInfo m, ...> { ... }
multiclass VPatBinaryV_VV<string intrinsic, string instruction, ...> { ... }
...
defm PseudoVADD : VPseudoVALU_VV_VX_VI<Commutable=1>;
```

llvm/lib/Target/RISCV/RISCVInstrInfoVPseudos.td:2160 / :4810 / :6111

(d) **LLVM intrinsic** 由 `defm vadd : RISCVBinaryAAX;(IntrinsicsRISCV.td:1199,1371)` 声明; (e) **前端 builtin** 由 `defm vadd : RVVIntBinBuiltinSet; (riscv_vector.td:759)` 声明。

值得强调：上游前端这层本身已经是生成的。clang/utils/TableGen/RISCVVEmitter.cpp 中的 RVVEmitter 后端从 riscv_vector.td 出发，分别产出 builtin 表、CodeGen 派发、sema 与 riscv_vector.h 头文件 (RVVEmitter::createHeader:407、createBuiltins:505、createCodeGen:573)。也就是说，上游真正的“人为知识”集中在后端：格式类的 `let Inst{}`、以及 VALU/VPseudo/VPat 这套深层多类库。auto-td 的发力点正是这一层（前端侧两者都已生成，auto-td 在 Clang 侧并不更优，见 §5）。

3 旧方法基线（对比的一侧）

把上节归纳成“端到端加一条 `vadd.vv` 需要做什么”：(1) 选/写格式类并把编码位手工誊进 `let Inst{}` (RISCVInstrFormatsV.td)；(2) 选对 ALU 多类并 `defm` 出指令记录 (RISCVInstrInfoV.td)；(3) `defm` 出 VPseudo 与 VPat (RISCVInstrInfoVPseudos.td)；(4) `defm` 出 LLVM intrinsic (IntrinsicsRISCV.td)；(5) `defm` 出 Clang builtin (riscv_vector.td)，其后前端四件套由 RVVEmitter 生成；(6) MC 编/解码由格式类的 `let Inst{}` 自动派生（上游已自动化）。

常见情形约触及 4-5 个 `defm`，但其真实成本是这些 `defm` 背后约六层类/多类的间接：作者必须先理解 `VALU_IV_V_X_I` $\rightarrow$ `VALU_IV_V/X/I` $\rightarrow$ `VALUVV` $\rightarrow$ `RVInstVV` 与 `VPseudo*/VPat*` 的层级，才能挑对复用点；一旦是库里没有的新形态（例如一条自定义逐元素函数指令），就得**新增/扩展格式类与多类机制**（格式类 .td、ALU 多类 .td、VPseudos.td、VPat、IntrinsicsRISCV.td、riscv_vector.td，乃至 per-CPU 调度模型 RISCVSched*.td），跨多个文件，文件与心智成本显著上升。换言之：编码位被人手誊写、形态被人手挑类——而这两件事，恰恰是最“机械”、最可由数据派生的部分。

4 YSX auto-td 系统设计（对比的另一侧）

4.1 权威数据分层

auto-td 把“加一条指令”拆成三种各有归属的事实。**opcode 数据库拥有编码位**：变量字段位置取自 `arg_lut.csv`，固定位取自 opcode 行内的 `msb..lsb=val` 语法。解析逻辑只做“把无等号的 token 当变量字段、有等号的当固定位”：

```
fields = tuple(part for part in parts[1:] if "=" not in part)
fixed_bits = tuple(part for part in parts[1:] if "=" in part)
field_ranges = tuple(OpcodeFieldRange(field, *arg_lut[field])
```

```

        for field in fields if field in arg_lut)
records[key] = OpcodeRecord(key, mnemonic, fields, fixed_bits, path, field_ranges)

```

llvm/lib/Target/YuShuXin/auto-td/tools/ysx_auto_td/opcodes.py:23-32

taxonomy 拥有共享的操作数/副作用形态（按类目，而非按指令）：它引用 opcode 的字段名（vd/vs1/vs2/vm）作为连接键，赋予角色/寄存器类/副作用——这些是 opcode 里没有的设计信息。下列为节选（省略 domain、default_surfaces 与各 false 副作用行）：

```

custom_float_unary:
  operation: fexp
  operands:
    outs: [{role: dest, field: vd, reg_class: VR}]
    ins:
      - {role: value, field: vs2, reg_class: VR}
      - {role: mask_policy, field: vm, operand: VMaskOp}
  effects: {implicit_uses: [VL, VTYPE]}
  sched: {semantic_class: vector_float_alu}

```

llvm/lib/Target/YuShuXin/auto-td/taxonomy/vector-alu.yaml:66-80 (*custom_float_unary*, 节选)。其中 sched 当前仅信息性，生成器恒置 hasNoSchedulingInfo=1。

一个 **YAML 拥有指令身份**（助记符、opcode 来源、spec_ref、特性要求，及可选的 pseudo/pattern/builtin 事实），且不含任何编码位（见 §4.3 禁词守护）。注意 spec_ref（如 tinyv.vector-alu.custom_float_unary）只是 taxonomy 类目的索引键（loader.py:184--185 取 rsplit('.')[1]），它不指向 riscv-isa-manual；isa-manual 当前未被任何工具或字段机器引用（见 §9）。

4.2 生成扁平记录（无逐指令嵌套类）

生成器把上述事实汇成一条单一共享基类 **RVInst** 的扁平 **def**——没有任何逐指令的派生类层级：

```

def _emit_instruction(instruction) -> list[str]:
    record_name = _record_name(instruction)          # "YSX_AUTO_" + 规范化助记符
    outs, ins, asm_operands = _operand_dags(instruction)  # 来自 taxonomy
    assignments = _assignments(instruction)           # 来自 opcode 编码位
    lines = [ f'def {record_name} : RVInst<{outs}, {ins}, "{mnem}"> {{',
              f'    let Predicates = [{_predicate(instruction)}];', ... ]
    # 恒发 hasSideEffects/mayLoad/mayStore/hasNoSchedulingInfo, 再逐位写 let Inst{...}
def _record_name(instruction):
    return "YSX_AUTO_" + source_key_from_mnemonic(instruction.mnemonic).upper()

```

llvm/lib/Target/YuShuXin/auto-td/tools/ysx_auto_td/emit_td.py:334-366（节选）

数据流与三处 CMake fan-out 见图 1。

4.3 守护：禁止把“已派生”的事实手写回 YAML

校验器禁止在指令 YAML 里夹带原始 TableGen 或拷贝来的编码事实，从源头保证“机械层只来自 opcode”：

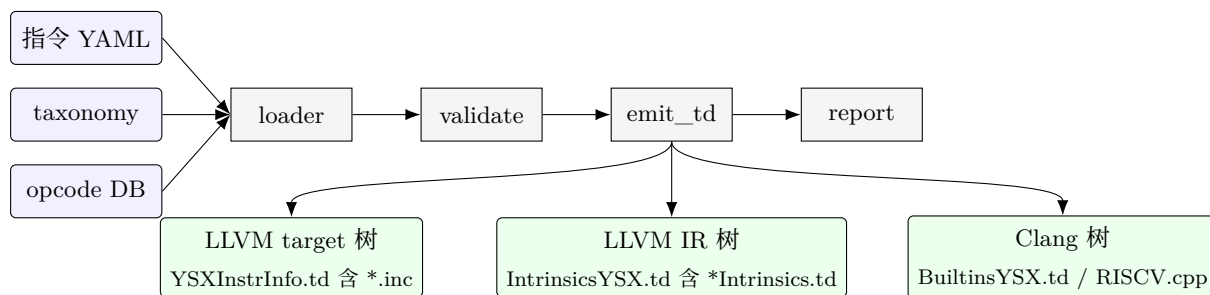


图 1: auto-td 数据流与三处 CMake fan-out: 指令 YAML + taxonomy + opcode DB 经 loader/-validate/emit_td/report, 扇出到 LLVM target、LLVM IR、Clang 三棵构建树。

```
FORBIDDEN = ("raw_td", "def : Pat", "RVInst", "VUnitStrideLoad",
             "VRED_", "VPseudo", "let Inst{", "bits<")
FORBIDDEN_FIELDS = {"raw_cpp", "encoding", "fixed_bits"}
```

llvm/lib/Target/YuShuXin/auto-td/tools/ysx_auto_td/validate.py:4-14

5 新旧对比分析（核心章）

图 2 给出旧路深层多类间接与新路扁平生成的对照；表 1 逐维展开。

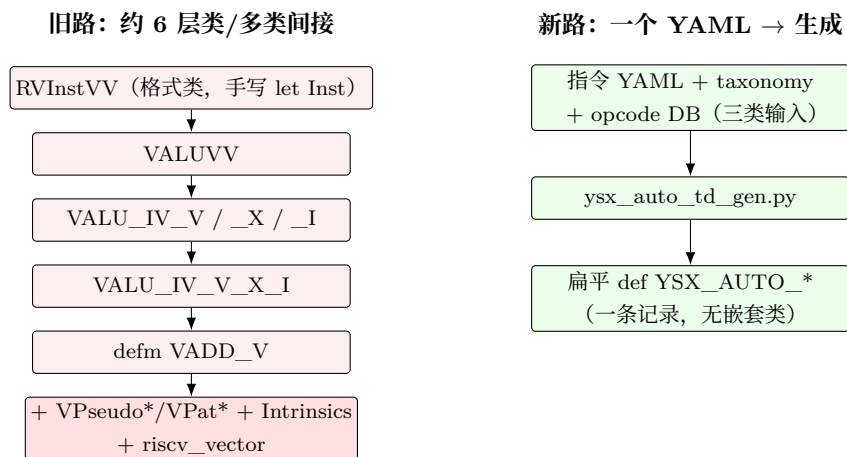


图 2: 旧路 vs 新路。旧路：作者须沿约六层类/多类挑对复用点；新路：复杂度集中在生成器，单指令呈一条扁平 def。

[†] 这是对比中最需诚实的一格：上游一条 defm PseudoVADD 会展开出完整的伪指令与模式矩阵，是真正的功能产物；auto-td 的扁平 def 每条只生成一条指令记录，YSXGenAutoTinyVPseudos.inc 当前为空（no generated records yet），相应矩阵尚未下降——因此本格不是“新路更省”，而是“新路尚未覆盖”。

命题的诚实分层判断。“上游 TableGen 的人为知识本可由 opcodes/isa-manual 解析获取”——

- 对机械层成立：RVInstVV 里那串 let Inst{31-26}=funct6 ... 6-0=OPC_OP_V (RISCVInstrFormatsV.td:107--125) 所誊写的，与 opcode 行 vadd.vv 31..26=0x00

维度	旧路（上游 RVV bring-up）	新路（YSX auto-td）
触及文件	常见约 4-5 个 defm（背后约六层类/多类）；新形态需跨多文件新增/扩展格式类与多类	一个 YAML；新形态再加一个 taxonomy 类目
TableGen 类机制	约 6 层类/多类间接（格式类 →ALU 多类 →VPpseudo→VPat）	零逐指令类；统一 RVInst 扁平 def
编码位来源	手写进格式类 let Inst{}	100% 派生自 opcode DB，YAML 禁含编码
操作数/副作用	手写进操作数类与 let mayLoad/...	100% 来自 taxonomy 类目（按字段名连接）
pseudo/pattern 矩阵 [†]	单条 defm 展开完整 LMUL/掩码/policy 矩阵	不生成：*Pseudos/*Patterns.inc 为注释 manifest
intrinsic/builtin 胶水	defm + RVVEmitter 生成前端四件套（含头文件）	codegen:true 时生成 builtin TD/intrinsic TD/CGBuiltin；不含用户头文件
MC 层	自 let Inst{} 自动派生	自生成记录 let Inst{} 自动派生（同机制）
ISel/lowering	标准 intrinsic 走声明式 VPat	手写 C++（全部 8 条 codegen 指令，见 §6）
评审面	跨多文件读 defm，核对挑类	读一个约 10 行 YAML（+ 可能一个类目）
复杂度所在	深层手维护的多类库	集中在一个 Python 生成器 + 声明式 taxonomy
不变量强制	N/A（手写即正道）	源端守护 + 测试注册门禁 (§7，限其覆盖形态)

表 1: 新旧逐维对比。

vm vs2 vs1 14..12=0x0 vd 6..0=0x57 是同一份信息。编码、操作数字段位置、汇编/反汇编，确实可由 opcode DB 立刻派生——auto-td 正是这么做的。

- 对语义/设计层不成立：操作数角色与寄存器类、mayLoad/mayStore、隐式 VL/VTYPE、ISel 语义、调度、intrinsic 类型、ABI——riscv-opcodes 里没有（连 RVInstVV 都还要手写 RVVConstraint），riscv-isa-manual 只有自然语言散文，无法靠解析立刻得到。
- 因此 auto-td 的创新是分离：机械层完全派生；设计层收敛为 taxonomy 这份集中、去重的残余人为知识（opcodes 给编码、taxonomy 给共享形态、YAML 给身份）+ 极少量算法性手写 C++。这份残余知识在指令共享形态时是次线性的（49 条 RVV 指令复用少数类目）；但对形态各异的厂商自定义指令，每条约需一个新 taxonomy 类目 + 手写下降，近似按条计（如 custom_float_unary 仅服务 vfexp 一条）。

6 端到端案例：yushuxin.vfexp

yushuxin.vfexp 是本切片中唯一的厂商自定义指令，用作“只往数据源加必要信息即可端到端”的存在性证明（非代表全部 50 条；其余 42 条只到记录 +MC 层）。

第一步，opcode 一行（与 riscv-opcodes 同语法；注意此行由本团队写入 ysx-opcodes，是“单源录入”而非外部权威）：

```
yushuxin.vfexp 31..26=0x2a vm vs2 19..15=0 14..12=0x1 vd 6..0=0x0b
```

third_party/ysx-opcodes/extensions/rv_xtinyv:1

第二步，一个 YAML 清单（无任何编码位）。其 header：字段是审计元数据、不生成指令

(生成器不产 .h，用户头文件手写，见第六步)；masked: true 与多 LMUL 当前尚未实现 (ISel 仅下降 v4f32 非掩码路径)：

```
mnemonic: yushuxin.vfexp
opcode_source: {repo: ysx-ops, extension: rv_xtinyv, key: yushuxin_vfexp}
spec_ref: tinyv.vector-alu.custom_float_unary
features: {required: [xtinyv]}
pseudos:
  matrix: {element_types: [f32], lmul: standard, masked: true, policy: llvm_default}
patterns:
  - {kind: intrinsic_to_pseudo, intrinsic: ysx.vfexp, operation: fexp}
builtin:
  header: ysx_vector.h
  names: [ysx_vfexp_v_f32m1]
  overloaded: false
  codegen: true
```

llvm/lib/Target/YuShuXin/auto-td/instructions/tiny-v/yushuxin_vfexp.yaml:1-13

第三步，运行生成器，得到真正的指令记录（由 emit_td.py:_emit_instruction 产出，记录名 YSX_AUTO_YUSHUXIN_VFEXP；落在构建树 YSXGenAutoTinyVInstrInfo.inc，committed 树中不存在任何手写版本；构建树行号随树而异，故以记录名引用）。以下为逐字生成内容：

```
// opcode-source: ysx-ops/rv_xtinyv/yushuxin_vfexp
def YSX_AUTO_YUSHUXIN_VFEXP : RVInst<(outs VR:$vd), (ins VR:$vs2, YSXAutoVMaskOp:$vm), "yushuxin
  ↪ .vfexp", "$vd, $vs2$vm", [], InstFormatR> {
  let Predicates = [HasStdExtXTinyV];
  let hasSideEffects = 0;
  let mayLoad = 0;
  let mayStore = 0;
  let hasNoSchedulingInfo = 1;
  let Uses = [VL, VTYPE];
  bit vm;
  bits<5> vs2;
  bits<5> vd;
  let Inst{31-26} = 0b101010;
  let Inst{25} = vm;
  let Inst{24-20} = vs2;
  let Inst{19-15} = 0b00000;
  let Inst{14-12} = 0b001;
  let Inst{11-7} = vd;
  let Inst{6-0} = 0b0001011;
}
```

由 emit_td.py 逐字生成；逐位对照：Inst{31-26}=0b101010 即 opcode 行 31..26=0x2a, 14-12=0b001 即 14..12=0x1, 6-0=0b0001011 即 6..0=0x0b——编码位机械派生，非手写。

对 builtin.codegen: true，生成器还产出 Clang builtin (vfexp_v_f32m1)、LLVM intrinsic (int_ysx_vfexp) 与 CGBuiltin 派发，分别被手写薄封装 BuiltinsYSX.td / IntrinsicsYSX.td / RISCV.cpp 引入。它替代了第 3 节的步骤 1-3 (选/写格式类誉编码、为新形态新增多类与 defm)

全部被“opcode 一行 + YAML 一份 + 跑生成器”取代；步骤 4-5 的 intrinsic/builtin TD 亦由 codegen:true 自动产出。

端到端测试链。 ysx_vfexp_v_f32m1 (手写头文件 ysx_vector.h) → __builtin_ysx_vfexp_v_f32m1 → llvm.ysx.vfexp → 生成记录。两个测试分别守住不同层：yushuxin-vfexp.c 守 C→IR (断言 call <4 x float> @llvm.ysx.vfexp) ; tinyv-builtins-asm.c 才是目标码证明(-emit-obj + llvm-objdump, // OBJ: yushuxin.vfexp 于该文件第 100 行)。

诚实的 gap (系统性, 非 vfexp 个例)。 intrinsic→ 机器下降对全部 8 条 codegen 指令均为手写 C++, 各自硬限于 v4i32/v4f32 固定形态, 且当前无任何生成的 DAG pattern:

```
case Intrinsic::ysx_vadd: { ... }
case Intrinsic::ysx_vsub: case Intrinsic::ysx_vmul:
case Intrinsic::ysx_vredsum: case Intrinsic::ysx_vfredsum:
case Intrinsic::ysx_vrgather: {
    if (...) { if (VT != MVT::v4f32) ... } else if (VT != MVT::v4i32) { ... }
}
case Intrinsic::ysx_vslideup: { if (...) || VT != MVT::v4i32) break; ... }
case Intrinsic::ysx_vfexp: { if (...) || VT != MVT::v4f32) break; ... } // :1195
```

llvm/lib/Target/YuShuXin/YSXISelDAGToDAG.cpp:1133-1206 (8 条 codegen 指令的手写下降, 节选)

*Pseudos/*Patterns.inc 是注释式审计 manifest、不是 DAG pattern; 真正的下降是上面这段手写 C++。ISel 硬限固定向量类型, 故 masked:true 与多 LMUL 当前是愿景, 尚未下降。用户头文件 clang/lib/Headers/ysx_vector.h (8 个手写 inline 封装) 整份手写。指令记录、MC 编码、intrinsic TD、Clang builtin TD 是真实且经测试的; 剩余的 vsetvli 插入、intrinsic 选择与公共头文件是有界的手写部分。

7 守护与威胁分析

审计发现一个 HIGH 级缺口: 承诺“新增指令必须经 auto-td”此前仅靠约定——validate.py 只读 YAML、从不读 .td, 且整套单测只在手动 unittest 下运行, 不门禁构建或 CI。据此补强(关系见图 3)。

(1) 源端守护: 新增 test_no_handwritten_instruction_td.py, 扫描 committed 顶层后端 .td, 对 def ... : RVInst* 形态的指令记录, 若 body 带 tiny-F/V 谓词或 tiny-FV 助记符且落在生成 include 之外即失败, 并断言“生成的 YSX_AUTO_* 记录集合 == YAML 声明的助记符集合”:

```
def _instruction_records(td_path): ... # 解析单行/块状两种 def (66-99)
for name, lineno, body in _instruction_records(td_path): # 判定循环 105-112
    has_predicate = any(pred in body for pred in TINY_FV_PREDICATES)
    mnemonic_hit = next((mn for mn in tiny_fv_mnemonics if f'"{mn}"' in body), None)
    if has_predicate or mnemonic_hit:
        offenders.append(f"{td_path}:{lineno}: def {name} ({reason})")
```

llvm/lib/Target/YuShuXin/auto-td/tests/test_no_handwritten_instruction_td.py:66-112 (节选)

守护的实际边界 (如实说明)。该词法扫描覆盖最直接、最可能的绕过路径——在顶层后端 .td

手写一条 `def ... : RVInst*` 指令记录。它当前不覆盖：经 `defm` 多类实例化、经非 `RVInst` 包装类派生、或置于子目录 `.td` 的指令记录，也不检查 `YSXISelDAGToDAG.cpp` 里的手写下降。这些更完整的闭合留给 §9 计划中的 `llvm-tblgen` 语义门。

(2) **边界哨兵**：在 `YSXInstrInfo.td:1751--1762` 用 `// YSX-AUTO-TD-BEGIN/END` 围住生成 `include`，守护断言哨兵区间内不得出现手写 `def`。(3) **数值/往返守护**：断言 `auto_full == YAML` 数、`retained_schema_gap == 0`；逐 `YAML` 校验声明的 `pseudo/pattern/builtin` 事实都出现在生成 `manifest` 中。(4) **注册门禁**：整套守护既注册为 `lit` 测试 (`auto-td-guards.test`，失败即让 `check-llvm/CI` 失败)，又注册为构建期目标，使普通 `ninja` 也门禁：

```
COMMAND ${Python3_EXECUTABLE} -m unittest discover
      -s ${CMAKE_CURRENT_SOURCE_DIR}/auto-td/tests -p "test_*.py" # :104
add_custom_target(YSXAutoTdGuards ALL DEPENDS ${YSX_AUTO_TD_GUARD_STAMP}) # :118
add_dependencies(YSXCommonTableGen YSXAutoTdGuards) # :119
```

llvm/lib/Target/YuShuXin/CMakeLists.txt:95-119 (节选)

验证：51 个 `auto-td` 单测全过；向顶层 `.td` 注入一条手写 `tiny-F def` 时守护按 `file:line` 报错并失败，移除后恢复；`YSXAutoTdGuards` 在构建期执行；覆盖率为 `auto_full:50`、`retained_schema_gap:0`。

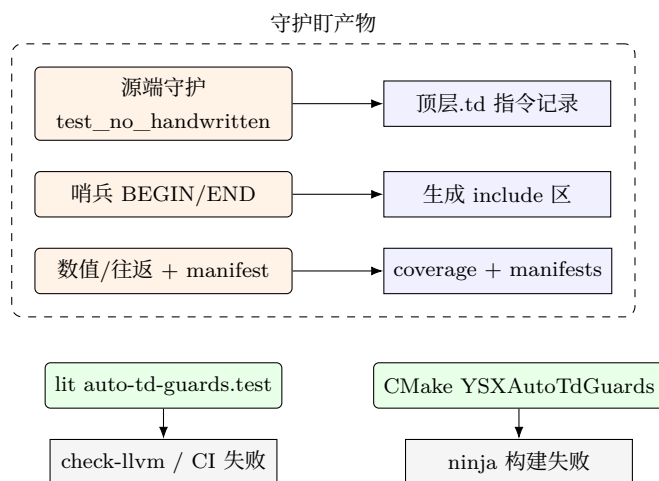


图 3: 守护关系：三类守护各盯一类产物，整套经 `lit` 与 `CMake` 双注册，分别门禁 `check-llvm/CI` 与本地 `ninja` 构建。

8 验证与覆盖

- 单测：`unittest discover` → 51 tests OK。
- 覆盖（以 `build/lib/Target/YuShuXin/auto-td/coverage.md` 为权威产物）：`auto_full:50`（15 `tiny-F` + 35 `tiny-V`）、`auto_with_structured_override:0`、`retained_schema_gap:0`；其中 8 条带 `builtin.codegen` (`validate.py` 白名单恰为这 8 类) 具备端到端路径，其余 42 条仅到记录 + MC 层。
- `lit`（标注所在层）：MC 层 `llvm/test/MC/YSX/{tinyf-auto-td.s, tinyv-auto-td.s,`

tinyv-invalid-disassemble.s};CodeGen 层 llvm/test/CodeGen/YSX/{auto-td-guards.test, tinyv-builtins-isel.ll, tinyf-isel.ll};端到端目标码 clang/test/CodeGen/YSX/tinyv-builtins-a (-emit-obj + llvm-objdump 往返) ——均通过。

- 局限：本轮验证停在生成/文本 + lit 层；生成的 TableGen 已经 lit 工具链消费，但未以 llvm-tblgen -print-records 做语义层门禁（列为 §9 未来工作）。

9 局限与未来工作

- **真实 DAG pattern 生成**：当前 *Patterns.inc 为注释 manifest，全部 8 条 codegen 指令的下降仍是手写 C++；可让生成器产出真正的 VPat 风格模式。
- **多 LMUL / 掩码 / 固定类型**：解除 v4i32/v4f32 硬限，按 pseudo matrix 下降。
- **tblgen 语义门**：增加一道 llvm-tblgen 守护，断言每个带 tiny-FV 谓词的指令记录名都以 YSX_AUTO_ 开头——这能闭合 §7 词法守护未覆盖的 defm/包装类/子目录/间接路径。
- **InstFormat 细化**：_inst_format (emit_td.py:399--407) 当前是粗启发式（向量类恒 InstFormatR），可做精确推断。
- **调度面**：taxonomy 的 sched：当前仅信息性（恒置 hasNoSchedulingInfo=1），可落地。
- **从 riscv-isa-manual 自动提炼语义**：现状机器解析仅 riscv-opcodes, isa-manual 当前未被任何工具或字段引用（仅作团队人审旁路参考）。把散文式语义半自动提炼进 taxonomy 是明确未来方向，需 NLP/结构化抽取，**不在本轮范围**。
- **明确不在范围**：完整 RVV 前端类型/可伸缩向量 C ABI、广义自动向量化。

10 结论

承诺“每条新 tiny-F/tiny-V 指令都从 opcode 源 + 一个 YAML 生成、无手写嵌套 TableGen 指令类”在当前 50 条切片中**实质达成**：committed .td 无任何 YSX_AUTO_* 或 tiny-FV 谓词记录，50 条仅经生成 include 进入构建，每条都是单一共享基类的扁平 def。补齐源端守护与测试注册后，这一承诺在**守护覆盖的形态内（顶层 .td 的 RVInst 族指令记录）**从约定升级为构建/CI 门禁；defm/包装类/间接路径的完整闭合由计划中的 tblgen 语义门接管。对照表明：声明式数据驱动显著降低单指令的工程与评审成本，把复杂度从“每指令深层间接”迁移到“集中式生成器 + 声明式 taxonomy”；代价是对生成器与数据源正确性的集中依赖、当前仍为手写的下降，以及尚未生成的 pseudo/pattern 矩阵。对“人为知识可由数据派生”的命题，诚实的回答是**分层成立**：机械层是，设计层否——而把两者分开，正是本工作的价值所在。

附录 A —50 条指令的 opcode 来源

15 tiny-F + 34 tiny-V 来自外部权威 riscv-opcodes (rv_f/rv_v) ; 1 条 yushuxin.vfexp 来自本团队维护的 ysx-opcodes (rv_xtinyv) 。8 条带 builtin.codegen (见 §8) 。权威覆盖产物为 build/lib/Target/YuShuXin/auto-td/coverage.md，源真值为 auto-td/instructions/{tiny-f,tiny-v}/*.yaml 的文件计数 (15 + 35)。

附录 B — 关键文件/行号索引

- 生 成 器: `auto-td/tools/ysx_auto_td/{opcodes.py:11--33, loader.py:184--185, validate.py:4--14, emit_td.py:334--366/399--407, report.py:5--25}`
- 数据: `taxonomy/vector-alu.yaml:66--80, instructions/tiny-v/yushuxin_vfexp.yaml:1--13, third_party/ysx-opcodes/extensions/rv_xtinyv:1`
- 后端接入与手写部分: `YSXInstrInfo.td:1751--1762, YSXISelDAGToDAG.cpp:1133--1206, clang/lib/Headers/ysx_vector.h`
- 守护: `test_no_handwritten_instruction_td.py:66--112, llvm/test/CodeGen/YSX/auto-td-guards.test.CMakeLists.txt:95--119`
- 端到端测试: `clang/test/CodeGen/YSX/{yushuxin-vfexp.c(C→IR), tinyv-builtins-asm.c (目标码:100) }`
- 上 游 对 比: `RISCVInstrFormatsV.td:107--125, RISCVInstrInfoV.td:612,1137, RISCVInstrInfoVPseudos.td:2160,4810,6111, IntrinsicsRISCV.td:1199,1371, riscv_vector.td:759, RISCVVEmmitter.cpp:407,505,573`

参考文献

1. LLVM Project 源码 (本仓库 `llvm/`、`clang/`)。
2. RISC-V `riscv-opcodes` (`third_party/riscv-opcodes`, 外部权威编码数据库)。
3. RISC-V ISA Manual (`third_party/riscv-isa-manual`, 已 vendored, 当前未被工具链机器引用, 仅作团队人审旁路参考)。
4. 本仓库设计/计划/实现文档: `docs/superpowers/{specs,plans,implementation,blog}/`。